



College of Management of Technology | Financial Engineering

Financial applications of blockchains and distributed ledgers

FINAL PROJECT

March 2026

Team size: 4-5 students

Deadline: 22 May 2026; 23:59 CET

Please submit your deliverables before the deadline via Moodle and email to:
dimitrios.karyampas@epfl.ch

Project Overview

Motivation

Decentralised exchanges built on Automated Market Maker protocols have fundamentally altered how financial liquidity is provided and consumed. Unlike centralised limit order books, AMMs such as Uniswap V3 rely on passive liquidity provision across discrete price ranges, and their execution quality is shaped by the distribution of that liquidity rather than by quote competition among active market makers.

Please address all questions below:

The Pool Under Study

All analysis in this project is conducted on a single, fixed pool:

Parameter	Value
Protocol	Uniswap V3
Pair	USDC / WETH
Fee tier	0.05% (tick spacing = 10)
Contract address	0x88e6A0c2dDD26FEEb64F039a2c41296FcB3f5640
Network	Ethereum Mainnet
Token0	USDC (6 decimals)
Token1	WETH (18 decimals)

Study Window

Students must define **at least a six-month historical observation window**, e.g. 1 October 2025 to 31 March 2026.

Project Structure

The project is organised into five core modules:

Module	Title	Grade weight
Module 1	On-Chain Data Extraction	20%
Module 2	Liquidity Distribution Analysis	20%
Module 3	Slippage Simulation and Execution Cost	20%
Module 4	Liquidity Provision Analytics	25%
Module 5	Dynamic Hedging of Impermanent Loss	15%

Overall Deliverables

You must submit all of the following:

#	Deliverable	Description
1	Codebase	A well-documented Python repository covering all 5.
2	Report	A research report addressing all questions. Figures in the report must be self-contained with axis labels, legends, and units.

Note: all on-chain data must be extracted directly via RPC calls to an Ethereum archive node (e.g. Alchemy, Infura, or QuickNode free tier is sufficient). The grader will verify that submitted code makes direct on-chain calls.

Module 1 — On-Chain Data Extraction

All quantitative analysis in this project depends on a single foundation: the ability to read and reconstruct the state of a live smart contract from raw blockchain data.

Students are required to extract all data directly from the Ethereum blockchain using RPC calls to an archive node. Students must also obtain access to an Ethereum archive node via a hosted RPC provider before beginning any extraction.

Required Outputs

The module must produce four clean, well-documented data files. Each file must be saved in Parquet format and accompanied by a data dictionary in the repository README describing every column, its type, and its unit.

Output 1 — `swap_events.parquet`

A record of every Swap event emitted by the pool during the six-month study window, decoded from raw event logs using the pool/contract ABI (Application Binary Interface – see <https://etherscan.io/address/0x88e6A0c2dDD26FEEb64F039a2c41296FcB3f5640#code>).

Required columns: `block_number`, `block_timestamp` (UTC), transaction hash, signed (i.e. positive when deposit into the pool, negative when withdrawal from the pool) USDC amount (raw and decimal-adjusted, i.e. human readable form – e.g. USDC 2'425), signed WETH amount (raw and decimal-adjusted), `sqrtPriceX96` after the swap, human-readable price (USDC per WETH), active liquidity at time of swap, tick after swap, trade direction (`buy_weth` or `sell_weth` from the taker perspective), and notional USD value.

*Note: for `block_timestamp` (UTC) make the call to derive it per unique block and **not** per swap.*

Output 2 — `mint_burn_events.parquet`

A complete record of every Mint and Burn event emitted by the pool since its deployment at block 12,376,729. The full history from deployment is required because the liquidity map at the start of the study window is the accumulated result of all prior LP actions.

Required columns: `block_number`, `block_timestamp` (UTC), transaction hash, event type (mint or burn), LP wallet address (owner), `tick_lower`, `tick_upper`, liquidity amount (raw), corresponding USDC and WETH amounts (raw and decimal-adjusted).

Note: Collect events are also required for Module 4 and should be downloaded during this same extraction pass to avoid a second full historical scan.

Output 3 — `liquidity_snapshots.parquet`

A reconstruction of the full tick-level liquidity map $L(\text{tick})$ at daily snapshots, one per day at a consistent time (the block closest to 00:00 UTC each day of the study window).

The liquidity map must be reconstructed by replaying Mint and Burn events from Output 2 up to each target block.

Required columns: `snapshot_block`, `snapshot_timestamp` (UTC), tick index (initialised ticks only), `liquidityNet` (signed), `liquidityGross`, `active_liquidity` (accumulated from minimum tick), `price_lower` (USDC/WETH at lower edge), `price_upper` (USDC/WETH at upper edge).

Validation requirement: For at least one snapshot, students must verify the reconstructed liquidity map against direct archive node calls to `pool.ticks()` for a sample of 10 initialised ticks. Results must match exactly. This validation must be documented in the written report.

Output 4 — `slot0_snapshots.parquet`

The pool price and active tick at each of the daily snapshot blocks, obtained via direct `eth_call` to the `slot0()` function at each target block number.

Required columns: `snapshot_block`, `snapshot_timestamp` (UTC), `sqrtPriceX96` (raw), `price_usdc_per_weth` (human-readable), current tick, `observation_index`, unlocked flag.

Consistency check: The price derived from `sqrtPriceX96` must be compared to the last recorded swap price before each snapshot block. The two should agree to within the tick spacing (0.01% for this pool). Any systematic discrepancy must be investigated.

Module 1 Deliverables

ID	Item	Description
D1.1	<code>data_extraction.py</code>	Fully documented script reproducing all four output files from scratch using only an RPC endpoint URL and the pool address as inputs
D1.2	Four data files	<code>swap_events.parquet</code> , <code>mint_burn_events.parquet</code> , <code>liquidity_snapshots.parquet</code> , <code>slot0_snapshots.parquet</code>
D1.3	Data dictionary	README section describing every column in every file: name, type, unit, and brief description
D1.4	Validation section	One page in the written report documenting the volume cross-check and the liquidity map spot-check, including results and any discrepancies found

Module 2 — Liquidity Distribution Analysis

Module 2 focuses on what does the liquidity landscape of this pool actually look like, and how does it evolve over time.

Task 2.1 — Liquidity Profile Visualisation

Using the daily snapshots from Module 1, students will produce liquidity profile charts where the x-axis is the USDC/WETH price (derived from tick index) and the y-axis is the active liquidity at each price level. The current pool price at each snapshot must be marked with a vertical line.

Students must produce: (a) static side-by-side profiles for three selected snapshots — the start of the study window, a day of high volatility, and the end of the window; and (b) a tiled time-series showing how the profile evolves across all snapshots.

Task 2.2 — TVL Computation and Decomposition

Students will compute TVL over time and decompose it into three components:

- In-range TVL: the USD value of liquidity in tick ranges that include the current price, i.e. liquidity that is actively earning fees and contributing to execution quality.
- Out-of-range TVL (above): the value locked in positions whose entire range is above the current price. These positions are 100% USDC and earn no fees.
- Out-of-range TVL (below): the value locked in positions whose entire range is below the current price. These positions are 100% WETH and earn no fees.

The decomposition must be computed at all daily snapshots and presented as a stacked area chart over time. The token amounts implied by each liquidity position must be derived using the Uniswap V3 virtual reserve formulas, which must be stated in the written report.

Task 2.3 — Liquidity Concentration Metrics

Students will compute two complementary concentration metrics at each of the daily snapshots.

Metric A — In-Range Liquidity Ratio (ILR)

The ILR(k) is defined as the fraction of total active liquidity located within a symmetric price band of $\pm k\%$ around the current price. Students must compute ILR for k in $\{0.1\%, 0.5\%, 1\%, 2\%, 5\%\}$ at each snapshot and plot all five series on the same time-series chart.

Metric B — Liquidity Herfindahl-Hirschman Index (L-HHI)

The L-HHI is the sum of squared liquidity shares across all initialised ticks: $HHI = \sum_i (L_i / \sum_j L_j)^2$. An L-HHI close to 1 indicates liquidity concentrated in very few ticks; a value close to 0 indicates near-uniform distribution. Students must compute L-HHI at each snapshot and overlay it with the ETH price on a dual-axis time-series chart.

Students must comment on the two metrics above and elaborate on their findings around liquidity concentration.

Module 2 Deliverables

ID	Item	Description
D2.1	liquidity_analysis.py	Script producing all figures from Module 1 data files
D2.2	Fig 2.1	Liquidity profile bar chart at three snapshots side by side (start, high-volatility day, end of window)
D2.3	Fig 2.2	Stacked area chart of TVL decomposition (in-range, above, below) over total number of days in the sample
D2.4	Fig 2.3	ILR(k) time series for all five bandwidths on one chart
D2.5	Fig 2.4	L-HHI time series overlaid with ETH price (dual y-axis)

Module 3 — Slippage Simulation and Execution Cost

Module 3 explores the relationship between liquidity distribution and trading cost. Students will simulate trade execution against the reconstructed pool state, and compute realised slippage from first principles.

Task 3.1 — The Uniswap V3 Swap Simulator

Students must implement a function that, given a reconstructed tick-level liquidity map at a specific snapshot block, the current price from slot0, a trade direction (buy WETH or sell WETH), and a trade size in USD notional, returns the executed average price, the price impact in basis points, and the realised slippage.

The simulator must implement the full Uniswap V3 step-by-step swap algorithm as described in the Uniswap V3 whitepaper. The algorithm proceeds in steps, each confined to a single tick interval where active liquidity is constant. At each step the simulator must: find the next initialised tick in the direction of the trade, compute how much input is consumed reaching that tick, either complete the trade within the current interval or cross into the next tick interval and update the active liquidity using liquidityNet.

The fee of 0.05% must be deducted from each input step before computing the output. The formulas governing the price and quantity relationships within each step are those from the Uniswap V3 whitepaper and must be stated and derived in the report.

Students must validate the swap simulator against a selection of real observed swaps from *swap_events.parquet* and document the results.

Task 3.2 — Simulation Grid

Students will run the simulator across the following grid:

- Trade sizes (both buy and sell): \$1K, \$10K, \$50K, \$100K, \$250K, \$500K, \$1M notional
- Snapshots: all daily snapshots from Module 1
- Directions: buy WETH (USDC in) and sell WETH (WETH in) at each size and snapshot

This produces a grid of 7 x # days x 2 simulated trades. For each trade, students must record: snapshot block and timestamp, direction, notional USD, average executed price, pool mid-price, price impact in basis points, slippage in basis points (price impact net of the 0.05% fee), and the number of tick boundaries crossed.

The full results must be saved as *simulated_trades.parquet*.

Task 3.3 — Price Impact Curve Analysis

Using *simulated_trades.parquet*, students will characterise how price impact scales with trade size.

For each of the daily snapshots, students will plot price impact in basis points against trade size on a log-log scale for both buys and sells. The median impact curve across all snapshots must be computed and overlaid with 10th and 90th percentile bands.

Task 3.4 — Effective Spread Estimation from Observed Swaps

As a complement to the simulation, students will estimate realised effective spreads directly from the observed swap data in *swap_events.parquet* without simulation.

For each observed swap, the effective spread is defined as: $ES_i = 2 * D_i * (p_{i_exec} - p_{i_mid}) / p_{i_mid}$, where $D_i = +1$ for a buy and $D_i = -1$ for a sell, p_{i_exec} is the execution price implied by the swap's amount0 and amount1, and p_{i_mid} is the pool price from slot0 at the block before the swap.

Swaps must be binned by trade size using the same seven size buckets from Task 3.2. The median effective spread per bin must be computed and compared to the simulated price impacts.

Module 3 Deliverables

ID	Item	Description
D3.1	swap_simulator.py	Standalone, well-documented implementation of the Uniswap V3 swap algorithm
D3.2	slippage_analysis.py	Script running the full simulation grid and producing all figures and regression tables
D3.3	<i>simulated_trades.parquet</i>	Full output of the simulation grid
D3.4	Fig 3.1	Simulator validation table: simulated vs. actual output, percentage error
D3.5	Fig 3.2	Price impact curves (bps vs. log trade size) with median and 10th/90th percentile bands
D3.6	Fig 3.3	Effective spread by trade size: simulated vs. empirically observed

Module 4 — Liquidity Provision Analytics

Modules 2 and 3 examined the pool from the perspective of a taker. Module 4 shifts to the perspective of the maker: the liquidity provider who deposits capital, earns fee income on every swap through their range, and bears the risk of impermanent loss when the price moves.

Task 4.1 — Constructing Representative LP Positions

Students will define five synthetic representative positions, all entered at the first daily snapshot block of the study window, all exited at the last daily snapshot block, and all sized at \$100,000 USD equivalent at the time of entry. The five positions are defined by their range width, centred on the current tick at entry:

Position	Price Range	Character
P1	$\pm 0.1\%$	Ultra-narrow, market-maker style
P2	$\pm 0.5\%$	Narrow, active LP style
P3	$\pm 2\%$	Medium, typical retail LP
P4	$\pm 10\%$	Wide, passive LP
P5	Full range	V2-equivalent, fully passive

For each position, students must compute the initial token amounts (x_0 WETH, y_0 USDC) deposited at entry and the corresponding liquidity value L , using the Uniswap V3 virtual reserve formulas. These formulas must be stated and derived in the written report. The derivation must show how to solve for L given the \$100,000 notional constraint.

Task 4.2 — Fee Income Computation

Fee income must be computed from first principles using the Uniswap V3 fee accumulator mechanism.

The core principle is that fees accrue to an LP position only when the current tick is within the position's range $[\text{tick_lower}, \text{tick_upper}]$. For each swap in *swap_events.parquet* where the current tick falls within a position's range, the LP's share of fees is proportional to their liquidity L divided by the total active liquidity at the time of the swap (as recorded in the Swap event's liquidity field).

Fee income is collected in both tokens: USDC fees on swaps where USDC is deposited into the pool, and WETH fees on swaps where WETH is deposited into the pool. Both must be converted to USD at the ETH price prevailing at the time of collection for aggregation.

This computation must be applied to all five positions across all daily snapshots to produce a cumulative fee income time series for each position.

Students must plot those cumulative fee income time series across those positions and comment.

Task 4.3 — Impermanent Loss Computation

Impermanent loss is defined as the difference between the mark-to-market value of the LP position and the value of a HODL portfolio holding the same initial quantities (x_0, y_0) without providing liquidity: $IL(t) = V_{HODL}(t) - V_{LP}(t)$. Under this sign convention, IL is a positive number when the LP position underperforms the HODL benchmark.

The HODL value is given by: $V_{HODL}(t) = x_0 * p_t + y_0$, where p_t is the ETH price at snapshot t . The LP principal value $V_{LP}(t)$ must be computed from the Uniswap V3 virtual reserve formulas, which produce different results depending on whether the current price is within the position range, above the range, or below the range. All three cases must be handled correctly and the formulas stated in the written report.

Both V_{HODL} and V_{LP} must be computed at all daily snapshots for all five positions and a figure of the $IL(t)$ time series for each of those positions should be presented in the report.

Finally a figure is to be generated with $(\text{Cumulative fee income}(t) - IL(t))$ for each position.

Module 4 Deliverables

ID	Item	Description
D4.1	lp_analytics.py	Script computing fee income, IL, and net P&L for all five synthetic positions and producing all figures
D4.2	Fig 4.1	Cumulative fee income time series for all five positions
D4.3	Fig 4.2	IL time series for all five positions
D4.4	Fig 4.3	Cumulative fee income minus Impermanent loss

Module 5 — Dynamic Hedging of Impermanent Loss

Module 4 established that impermanent loss is a real, measurable, and sometimes substantial cost for LPs. The natural next question is: can IL be hedged, and at what cost?

Hedging Instrument: Hyperliquid ETH Perpetual Futures

Assume the hedging instrument is the ETH perpetual futures contract on Hyperliquid. Hyperliquid is a decentralised exchange running on its own Layer 1 blockchain with a fully on-chain order book. Students must retrieve historical data via Hyperliquid API.

Task 5.1 — Theoretical Framework

5.1.1 — LP Payoff Shape and the Short Gamma Interpretation

Students must plot the payoff of the LP position as a function of terminal ETH price, for each of the five positions defined in Module 4, ignoring fee income. The plot must cover a price range from 0.5 to 1.5 times the entry price.

Students must plot and comment on the payoff shape and its similarity to well-known option payoffs.

5.1.2 — Derivation of LP Delta

Students must derive the delta of the LP position — the first derivative of V_{LP} with respect to the ETH price p — analytically from the Uniswap V3 virtual reserve formulas. The derivation must cover all three price cases: within range, above range, and below range.

5.1.3 — Gamma

Students must derive the gamma of the LP position — the second derivative of V_{LP} with respect to p . The sign and magnitude of gamma must be connected to LP position width.

Task 5.2 — Data Collection from Hyperliquid

Hourly Price Candles

Students must collect hourly OHLCV candles for the ETH perpetual on Hyperliquid for the full six-month study window.

The data must be saved as *perp_prices.parquet* with the candle close price used as the mark price in all subsequent hedge P&L calculations.

Hourly Funding Rates

Students must collect all funding rate records for the ETH perpetual on Hyperliquid for the full study window.

The `fundingRate` field returned by the API is the hourly rate. The funding payment per hour for a short position of ETH futures is: $\text{funding_pnl} = \text{position_size_ETH} * \text{oracle_price} * \text{funding_rate}$.

When `fundingRate` is positive (longs pay shorts), a short position receives funding. When `fundingRate` is negative, a short position pays funding.

The data must be saved as *funding_rates.parquet*. Students must produce a chart showing the hourly funding rate and cumulative funding rate over the study window alongside the ETH price, to characterise the funding environment faced by a short hedger during the chosen period.

Task 5.3 — Delta Hedging Backtest

Strategy Design

Students will backtest a discrete delta-hedging strategy for each of the five LP positions from Module 4. The strategy takes a short position in Hyperliquid ETH perpetual with size equal to $|\Delta_{LP}|$ ETH, where Δ_{LP} is the LP delta derived in Task 5.1.2. For the purpose of the funding formula defined in Task 5.2, `position_size_ETH` always denotes this absolute size as a positive number; the short direction is already implied by the formula's sign convention. The hedge is held static between rebalancing times, during which the net delta drifts due to gamma exposure.

Three rebalancing frequencies must be tested: hourly (1h), 4-hourly (4h), and daily (24h). This gives 5 positions x 3 frequencies = 15 strategy variants.

P&L Accounting

For each strategy variant and at each hourly time step, students must compute and accumulate the following P&L components:

- Hedge P&L: the mark-to-market gain or loss on the short perpetual position since the last time step. A short position gains when the ETH price falls and loses when it rises.
- Funding P&L: the funding payment is $|\Delta_{LP}| * \text{oracle_price} * \text{funding_rate}$, with sign as defined in Task 5.2 — positive when `fundingRate` > 0 (short receives), negative when `fundingRate` < 0 (short pays).
- Trading fees: deducted at each rebalancing event. Assume a fee equal to 0.045% per trade on notional value.

At each time step, students must also record: ETH price, LP principal value, HODL value, gross IL (unhedged), hedge P&L (cumulative), net hedge P&L (hedge P&L + funding P&L - trading fees), residual IL (gross IL minus net hedge P&L), LP fee income (from Module 4), and net position P&L (LP fee income minus residual IL minus net hedge cost). Store those in *hedge_results.parquet*.

Students must plot the results and comment accordingly.

Task 5.4 — Limitations and Extensions

Students must write a critical discussion addressing the following four limitations of the delta-hedging strategy:

- Residual gamma risk
- Funding rate risk
- Minimum LP position size for which the delta-hedging strategy is economically viable

- Alternatives of hedging via options and what necessary assumptions to make this effective (e.g. options availability and liquidity, etc.)

Module 5 Deliverables

ID	Item	Description
D5.1	<i>perp_prices.parquet</i>	Hourly OHLCV candle data from Hyperliquid ETH perp for the full study window
D5.2	<i>funding_rates.parquet</i>	Hourly funding rate records from Hyperliquid for the full study window
D5.3	<i>hedge_backtest.py</i>	Fully documented implementation of the delta-hedging backtest for all 15 strategy variants
D5.4	<i>hedge_results.parquet</i>	Full hourly backtest output for each strategy variant
D5.5	Fig 5.1	Hedging strategy results